Software Engineering Lab

Lab 26

Contract Testing

Consumer-Driven Contracts & CI/CD Deployment Gates

Lukman / lukman-ss

Masalah Integrasi Microservices: Rapuhnya E2E Test

End-to-end testing lambat, flaky, dan mahal untuk mendeteksi breaking schema changes

MASALAH E2E TESTING

Membutuhkan seluruh ekosistem service dan database aktif bersamaan di staging
Flaky network, timeout, dan latency test lingkungan memperlambat pipeline CI/CD
Feedback loop lambat: pengembang menunggu berjam-jam hanya untuk cek kompatibilitas DTO

PARADOKS MOCK SERVICE

Unit test consumer dengan mock lokal selalu lolos karena asumsi schema statis
Saat provider mengubah casing enum atau nama field di produksi, consumer langsung crash
Mock yang dirawat manual cepat usang dan menciptakan ilusi keamanan palsu

Consumer Mobile App membutuhkan kontrak schema minimal dari OrderService

Provider melakukan perubahan tanpa koordinasi yang melanggar kontrak DTO

Dibutuhkan deployment gate otomatis berbasis CDC tanpa dependensi external runtime

Mocks drift into false confidence; contract tests turn consumer assumptions into executable provider guardrails.

Anatomi Consumer-Driven Contracts: Consumer Menentukan Ekspektasi

Bukan provider yang mendikte payload, melainkan kebutuhan nyata consumer yang didaftarkan

```
Request: HTTP Method GET /v1/orders/ORD-123
Provider State: Order ORD-123 exists and is IN_PROGRESS
Response: Status 200 OK, Content-Type JSON, body dengan id, status, customer.name, total
```

Consumer hanya mencatat subset field yang benar-benar dibaca oleh kodenya

Field provider yang tidak dikonsumsi (misal internal notes) diabaikan dalam kontrak

Provider bebas berevolusi pada field yang tidak mengikat consumer mana pun

Provider-driven: Provider merilis OpenAPI spec raksasa, consumer dipaksa mengikuti

Consumer-driven: Consumer merilis kontrak kebutuhan, provider membuktikan kepatuhan

Hasil: Hubungan dependensi API menjadi eksplisit, terukur, dan dapat diaudit di CI

The consumer defines the contract, but the provider proves compliance before shipping.

Skenario Nyata: Tiga Mutasi Payload Pemicu Downtime

Perubahan sederhana di backend merusak parsing JSON aplikasi mobile

MUTASI 1: CASING ENUM STATUS

Backend mengubah 'IN_PROGRESS' menjadi huruf kecil 'in_progress' untuk standardisasi
Client mobile memiliki strict enum parser: status tidak dikenali memicu fatal error
User tertahan di layar loading tanpa pesan kesalahan yang informatif

MUTASI 2: FIELD RENAME CUSTOMER

Backend merapikan schema: field customer.name diubah menjadi customer.full_name
Parser consumer mengharapkan customer.name tidak boleh string kosong
Hasil: unmarshal error 'contract violation: customer.name is missing'

Backend mengirim total dalam bentuk string '150000' daripada integer 150000

Go json.Unmarshal gagal melakukan deserialisasi string ke int64

Kerusakan terjadi seketika di level deserializer sebelum domain logic dijalankan

Small unannounced payload changes cause catastrophic client-side deserialization outages.

CI/CD Deployment Gate: Can I Deploy?

Memindahkan verifikasi kompatibilitas dari staging runtime ke build pipeline

PIPELINE CONSUMER

Menjalankan unit test client dengan mock provider
Membangkitkan file kontrak JSON (MobileApp -> OrderService)
Mempublikasikan kontrak ke artifact store atau git repo

PIPELINE PROVIDER

Menjalankan server provider di lingkungan in-memory ephemeral
Menjalankan CDC Verifier membaca kontrak consumer aktif
Gate check: Jika verification gagal -> CI Provider FAIL -> Deployment BLOCKED

Tidak butuh staging environment yang selalu online 24/7

Eksekusi verifikasi selesai dalam milidetik (0.00s pada Lab 26)

Breaking change tertahan sebelum binary masuk ke container registry

Verify consumer contracts at build time so broken payloads never reach the cluster.

Arsitektur Lab 26: Desain Sistem Tanpa Dependency Daemon

Implementasi CDC murni Go standard library tanpa binary external pact-ruby

STRUKTUR MODUL

internal/model: Definisi DTO OrderResponseV1, Breaking, V2
internal/consumer: MobileOrderClient & GenerateMobileContract()
internal/provider: ProviderV1, ProviderBreaking, ProviderDual handlers
internal/contract: Verifier engine dengan recursive schema diffing

ALASAN DESAIN PURE GO

Menghilangkan dependensi cgo, daemon broker eksternal, atau CLI binary berat
Dapat dijalankan instan via go test -v ./... dan go run ./cmd/demo
Membuktikan semantik CDC secara murni, transparan, dan mudah dipahami

Verifier stateless menggunakan http.Client Go thread-safe

Uji konkurensi 20 goroutine paralel lolos uji race detector tanpa data race

Dapat diverifikasi ulang dengan perintah go test -race ./...

Pure standard-library architecture isolates contract mechanics from tooling friction.

Consumer Code: Mendefinisikan Kontrak Minimal

Mobile consumer mendefinisikan ekspektasi HTTP, headers, dan struktur response

```
func GenerateMobileContract() *contract.Contract {
    return &contract.Contract{
        Consumer: 'MobileApp', Provider: 'OrderService',
        Interactions: []contract.Interaction{{
            Request: contract.RequestDefinition{Method: 'GET', Path: '/v1/orders/ORD-123'},
            Response: contract.ResponseDefinition{Status: 200, ...},
        }},
    }
}
```

```
id: 'ORD-123'
status: 'IN_PROGRESS'
customer: { name: 'Budi Santoso' }
total: json.Number('150000')
```

Consumer mengabaikan field provider seperti 'notes' atau 'customer.id'

Menggunakan json.Number untuk membedakan integer vs string secara eksak

Mendokumentasikan provider state agar provider dapat menyiapkan data uji

Express only what the client consumes; ignore all ancillary provider attributes.

Mobile Client: Parsing Ketat Terhadap Pelanggaran Kontrak

Client mobile memvalidasi keberadaan field wajib dan konsistensi enum

```
if err := json.Unmarshal(body, &raw); err != nil {
    return nil, fmt.Errorf('failed to parse contract schema: %w', err)
}
if raw.Customer.Name == '' {
    return nil, fmt.Errorf('contract violation: customer.name is missing')
}
if raw.Status != 'IN_PROGRESS' && raw.Status != 'COMPLETED' {
    return nil, fmt.Errorf('contract violation: unknown status %q', raw.Status)
}
```

Jika customer.name hilang, client langsung mengembalikan error spesifik
Jika status berhuruf kecil 'in_progress', client menolak memproses pesanan
Mencegah nilai null merembet ke lapisan UI/domain mobile app
Aturan validasi inilah yang wajib tercermin pada kontrak CDC
CDC menjamin provider tidak merilis response yang melanggar aturan ini
Saling melengkapi: client strict, CDC memvalidasi provider sebelum deploy

Strict client validation protects app runtime; CDC guarantees the provider respects those rules.

CDC Verifier: Eksekusi Request & Validasi Status Code

Mengeksekusi HTTP interaction terhadap URL provider yang sedang berjalan

```
targetURL := strings.TrimRight(baseURL, '/') + interaction.Request.Path
req, err := http.NewRequest(interaction.Request.Method, targetURL, nil)
resp, err := v.Client.Do(req)
if resp.StatusCode != interaction.Response.Status {
    result.Passed = false
    result.Errors = append(result.Errors, fmt.Sprintf('status code mismatch...'))
}
```

Verifier memeriksa header Content-Type secara case-insensitive
Mencegah response text/html atau raw string saat consumer meminta JSON
Memastikan response HTTP valid sebelum masuk ke deserialisasi JSON
Menggunakan ephemeral httptest.Server di dalam proses testing
Tanpa roundtrip jaringan luar — selesai dalam 0.00 detik
Ideal untuk dimasukkan ke dalam setiap pull request dan pre-commit check

Run the interaction against a live ephemeral provider instance and assert HTTP semantics.

Algoritma Diffing: Validasi Recursive Subset Schema

Membandingkan struktur nested JSON hanya pada field yang diminta consumer

```
func diffValues(path string, expected, actual interface{}) []string {
    for key, expVal := range expMap {
        actVal, exists := actMap[key]
        if !exists { diffs = append(diffs, 'missing expected field') }
        diffs = append(diffs, diffValues(childPath, expVal, actVal)...)
    }
}
```

Membedakan json.Number (angka) dari string JSON secara presisi

Memverifikasi kecocokan tipe via reflect.TypeOf()

Mendeteksi perubahan nilai enum seperti 'IN_PROGRESS' vs 'in_progress'

Provider memiliki field 'notes' dan 'customer.id' yang tidak ada di kontrak

diffValues mengabaikan field ekstra tersebut tanpa menghasilkan error

Memungkinkan provider menambahkan fitur baru tanpa merusak consumer lama

Validate subset presence and exact types; tolerate unrequested provider fields.

Verifikasi Provider V1: Memenuhi Seluruh Kontrak

Server V1 mengirimkan DTO asli yang sesuai dengan harapan MobileApp

```
ID: 'ORD-123'
Status: 'IN_PROGRESS' (uppercase string)
Customer: { ID: 'cust-100', Name: 'Budi Santoso' }
Total: 150000 (int64)
Notes: 'Internal note not used by mobile app'
```

Status Code: 200 OK — MATCH
Header Content-Type: application/json — MATCH
Payload schema: 4/4 field consumer ditemukan dengan tipe data yang tepat
Field 'notes' dan 'customer.id' diabaikan secara aman
Verification result: Passed = true, Errors = 0
CI Deployment Gate: ALLOWED — Siap dideploy ke cluster produksi
Eksekusi test TestProviderV1_ContractVerification_Success: PASS (0.00s)

When expected subset matches completely, deployment gate turns green.

Breaking Provider: Terdeteksi & Diblokir di CI Gate

Tiga perubahan schema langsung memicu kegagalan verifikasi kontrak

```
[Stage 3] Running Breaking Provider Contract Verification:
Result: BLOCKED! Breaking changes detected before deployment:
  1. path 'total': type mismatch (expected 150000 [json.Number], got 150000 [string])
  2. path 'status': value mismatch (expected 'IN_PROGRESS', got 'in_progress')
  3. missing expected field 'customer.name'
CI Deployment Gate: PREVENTED PRODUCTION OUTAGE.
```

Total: tipe bermutasi dari integer ke string

Status: casing bermutasi dari UPPERCASE ke lowercase

Customer.name: nama field direname menjadi customer.full_name

Exit code non-nol menghentikan build runner CI/CD

Engineer backend langsung melihat daftar field yang melanggar kontrak

Produksi tetap aman: tidak ada deployment versi cacat ke live server

Contract test failures pinpoint exact field diffs before bad code reaches production.

Taksonomi Breaking Changes pada REST API

Klasifikasi perubahan payload yang memicu kegagalan consumer

Tipe Mutasi	Contoh Perubahan	Dampak Consumer
Field Rename	name -> full_name	Field missing / null
Type Mutation	int 150000 -> string '150000'	Deserialization crash
Enum Value Change	IN_PROGRESS -> in_progress	Validation/Logic error
Field Removal	Menghapus field total	Missing required field
HTTP Status Change	200 OK -> 202 Accepted	Client error branch

Menambahkan field baru di level root atau child (additive changes)
Menambahkan endpoint HTTP baru tanpa mengubah endpoint lama
Memperluas toleransi input request pada provider
Dengan jumlah consumer yang cukup, semua perilaku API yang dapat diamati akan menjadi dependensi
CDC mendokumentasikan observasi tersebut secara eksplisit dalam kode

Know what breaks: renames, type mutations, and enum alterations break clients; additions do not.

Strategi Evolusi API: Dual DTO & Parallel Routing

Memperkenalkan fitur baru tanpa merusak kontrak consumer lama

```
if strings.HasPrefix(r.URL.Path, '/v1/orders/') {
    respV1 := model.OrderResponseV1{ ... }
    json.NewEncoder(w).Encode(respV1)
}
if strings.HasPrefix(r.URL.Path, '/v2/orders/') {
    respV2 := model.OrderResponseV2{ ... }
    json.NewEncoder(w).Encode(respV2)
}
```

GET /v1/orders/{id} -> Mengembalikan schema V1 (kontrak MobileApp tetap terpenuhi)
GET /v2/orders/{id} -> Mengembalikan schema V2 (fitur baru: full_name, string total, currency)
Backend melayani kedua endpoint secara simultan dari domain model yang sama
ProviderDual lolos verifikasi kontrak MobileApp V1 dengan 0 error
Mobile App dapat terus beroperasi tanpa migrasi tergesa-gesa
Web / client baru dapat langsung mengadopsi schema V2

Evolve via parallel endpoints: satisfy existing contracts on V1 while innovating on V2.

Pola Expand–Contract: Migrasi Tanpa Downtime

Tiga fase evolusi API untuk mematikan schema lama dengan aman

FASE 1: EXPAND

Provider menambahkan endpoint /v2 atau field baru
Menjaga endpoint /v1 tetap beroperasi melayani kontrak aktif
CI gate memverifikasi kontrak lama tetap 100% compliant

FASE 2: MIGRATE CONSUMERS

Consumer secara bertahap memperbarui kodenya ke schema V2
Setiap consumer merilis kontrak baru (V2 interaction) ke broker/repo
Saat semua consumer telah migrasi, kontrak V1 menjadi tidak memiliki consumer

Provider mengecek broker: tidak ada consumer yang terikat pada kontrak V1

Endpoint /v1 dihapus secara aman tanpa risiko breaking change

Evolusi selesai tanpa pernah sekalipun memicu downtime

Expand new capabilities, migrate consumers independently, contract old paths safely.

Hasil Eksekusi Test: Lolos Unit, Integration & Race Detector

Seluruh rangkaian test lolos 100% tanpa memory leak atau race condition

```
=== RUN   TestConsumerContractGeneration -> PASS (0.00s)
=== RUN   TestProviderV1_ContractVerification_Success -> PASS (0.00s)
=== RUN   TestProviderBreaking_ContractVerification_Fails -> PASS (0.00s)
=== RUN   TestProviderDual_ContractVerification_Success -> PASS (0.00s)
=== RUN   TestConcurrentContractVerification -> PASS (0.01s)
=== RUN   TestVerifier_HeaderValidation_And_ErrorBranches -> PASS (0.00s)
=== RUN   TestProviderDual_V2Endpoint_DirectAssertion -> PASS (0.00s)
PASS: ok labs/26-contract-testing/tests 0.390s
```

Perintah: go test -race ./...

Hasil: ok labs/26-contract-testing/tests 1.425s

20 goroutine paralel memverifikasi kontrak tanpa konflik data

7 test scenario mencakup branch sukses, gagal, header, dan concurrent

TestProviderBreaking membuktikan minimal 3 error ditangkap secara eksak

Determinisme penuh: tidak ada sleep timer atau flakiness

Deterministic verification completes in milliseconds, clean under Go race detector.

Perbandingan Metode Pengujian Integrasi

Mengevaluasi Unit Test, End-to-End Test, dan Contract Test

Karakteristik	Unit Test (Mock)	End-to-End Test	Contract Test (CDC)
Kecepatan Eksekusi	Sangat Cepat (<1s)	Sangat Lambat(mnt)	Sangat Cepat (ms)
Kestabilan (Flaky)	Nol (Stabil)	Tinggi (Flaky)	Nol (Stabil)
Cakupan Integrasi	Tidak Ada (Palsu)	Penuh	Format & Semantik
Kebutuhan Lingk.	In-Process	Multi-Server/DB	In-Process Ephemeral
Deteksi Breaking	Gagal	Terlambat	Tepat Waktu (CI Gate)
Biaya Pemeliharaan	Rendah	Sangat Tinggi	Rendah

CDC mengisi gap kritis antara unit test yang terisolasi dan E2E test yang lambat

Memberikan keyakinan integrasi setara E2E dengan biaya eksekusi setara unit test

Paling efektif diterapkan pada arsitektur microservices terdistribusi

Contract tests berada di tengah piramida: di atas unit tests, di bawah E2E

Mengurangi kebutuhan suite E2E hingga lebih dari 70%

Contract tests deliver integration-level confidence at unit-test speed and stability.

Standar Industri: Ekosistem Pact & Pact Broker

Bagaimana prinsip lab ini diaplikasikan pada skala enterprise

KOMPONEN PACT FRAMEWORK

Pact Specification: Standar formal JSON untuk mendefinisikan interaksi HTTP dan message
Pact Language SDKs: pact-go, pact-js, pact-jvm, pact-python
Pact Broker: Server sentral yang menyimpan dan memvalidasi matriks versi kontrak

FITUR CAN-I-DEPLOY

CLI command: pact-broker can-i-deploy --pacticipant OrderService --version git-sha
Memverifikasi apakah versi provider kompatibel dengan semua consumer di environment target
Menjadi penentu exit-code pada pipeline CD sebelum rilis ke Kubernetes/Cloud

Lab 26 mengimplementasikan esensi verifikasi Pact murni dalam Go standard library

Pola diffing, provider states, dan CI gate yang dipelajari di sini 100% kompatibel secara konseptual

Memahami prinsip inti terlebih dahulu sebelum mengadopsi tooling enterprise

Master the CDC mechanics in pure code before scaling to enterprise Pact Broker workflows.

Trade-off & Biaya Penerapan Contract Testing

CDC bukan pengganti seluruh jenis pengujian: kenali batasannya

BIAYA & TANTANGAN

Disiplin Organisasi: Membutuhkan kolaborasi erat antara tim consumer dan tim provider
Manajemen Provider State: Menyiapkan data fixture agar provider merespons sesuai kontrak
State Setup Overhead: Endpoint mock/helper untuk seeding database sebelum verifikasi

APA YANG TIDAK DIUJI CDC

Performance & Latency: Tidak mengukur throughput, p99 latency, atau beban DB
Deep Business Logic: Tidak memverifikasi kalkulasi internal akuntansi yang kompleks
Security & Authorization: Tidak menggantikan vulnerability scanner atau penetration test

Fokuskan CDC pada batas kontrak DTO dan semantik interaksi API

Gabungkan dengan load testing (Lab 15) dan unit testing domain logic

Otomatiskan regenerasi kontrak saat consumer test dijalankan di local

Contract tests guard boundary semantics; business logic and performance still need their own tests.

Batasan Implementasi Lab 26

Asumsi penyederhanaan yang diambil untuk menjaga fokus pembelajaran

BATASAN 1: PURE GO RUNNER

Menggunakan verifier kustom berbasis Go net/http, bukan binary resmi Pact
Menghilangkan ketergantungan runtime ruby daemon untuk kemudahan eksekusi lokal
Algoritma diffing mencakup subset JSON, primitive types, dan exact string/number

BATASAN 2: IN-MEMORY BROKER

Kontrak dioper secara in-memory/in-process tanpa server Pact Broker eksternal
Pada produksi enterprise, kontrak disimpan di Pactflow atau dedicated server
Pola data flow tetap identik: consumer produce -> provider verify

Provider state disimulasikan melalui deterministic endpoint routing (/v1/orders/ORD-123)

Tidak mengimplementasikan state setup callback endpoint dinamis

Cukup untuk membuktikan deteksi kegagalan pada 3 jenis breaking changes utama

Understand the deliberate simplifications: pure Go eliminates setup toil while proving the core model.

Panduan Keputusan: Kapan Menggunakan CDC?

Kriteria arsitektur dan organisasi yang paling diuntungkan oleh contract testing

SANGAT DISARANKAN KETIKA

Banyak tim microservices independen yang merilis software dengan siklus berbeda
Aplikasi Mobile / Frontend SPA yang terikat pada backend API pihak ketiga
Lingkungan staging lambat, tidak stabil, atau memakan biaya cloud sangat mahal
Regresi API sering terjadi karena perubahan schema yang tidak sengaja

KURANG COCOK KETIKA

Monolith tunggal: seluruh pemanggilan adalah function call internal in-process
API publik eksternal dengan ribuan konsumen tidak dikenal (gunakan schema validation/OpenAPI)
Tim sangat kecil (1–2 orang) di mana komunikasi verbal jauh lebih murah daripada tooling

Semakin tinggi jumlah dependensi antar-layanan, semakin besar ROI dari contract testing

Memangkas waktu koordinasi rilis antar-tim dari harian menjadi otomatis di CI

Adopt CDC when cross-team microservice deployments slow down due to integration fear.

Checklist Adopsi CDC di Pipeline Produksi

8 langkah taktis mengimplementasikan deployment gate di organisasi

CHECKLIST PERSIAPAN & CONSUMER

[ ] Identifikasi batasan service dan consumer yang paling kritis (misal Mobile App)
[ ] Buat consumer test yang menghasilkan file kontrak interaksi minimal
[ ] Setup publish kontrak ke Pact Broker atau repository terpusat pada pipeline consumer

CHECKLIST PROVIDER & GATE

[ ] Implementasikan provider verification step di CI provider sebelum build container
[ ] Tambahkan automated provider state handler untuk menyiapkan mock DB
[ ] Pasang can-i-deploy gate untuk memblokir merge PR yang melanggar kontrak

[ ] Terapkan pola Expand-Contract saat memutasi schema DTO

[ ] Pantau utilisasi kontrak lama sebelum mematikan endpoint deprecated

Eight concrete steps turn integration anxiety into automated CI/CD confidence.

Tiga Prinsip Inti Contract Testing

Fondasi filosofis untuk membangun integrasi layanan yang tangguh

PRINSIP 1: MINIMAL SUBSET

Konsumen hanya meminta apa yang benar-benar digunakan
Mencegah pengikatan yang terlalu ketat (tight coupling) pada atribut yang tidak relevan
Memberi ruang bagi provider untuk berevolusi secara independen

PRINSIP 2: VERIFIKASI PADA BUILD TIME

Validasi terjadi sebelum artifact masuk ke container image
Menghilangkan ketergantungan pada runtime staging environment yang rapuh
Feedback loop instan bagi engineer saat masih berada di konteks kode

Perubahan selalu bersifat aditif melalui dual routing atau new optional fields

Jangan pernah mengubah tipe atau merename field yang terikat kontrak aktif

Migrasi konsumen secara asinkron sebelum mematikan jalur lama

Minimal subsets preserve provider agility; build-time gates eliminate runtime integration dread.

Ringkasan Lab 26: Contract Testing

Contract tests turn consumer assumptions into executable provider guardrails at build time.

HASIL EKSEKUSI UTAMA

Provider V1 lolos verifikasi kontrak consumer MobileApp secara instan

Breaking changes (enum, rename, primitive mutation) terdeteksi & diblokir di CI gate

Dual Provider (V1 + V2) mendemonstrasikan evolusi schema yang aman dan kompatibel

REPOSITORY

https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/26-contract-testing

Lukman / lukman-ss

MENTAL MODEL INTI

Contract tests turn consumer assumptions into executable provider guardrails.

Verify minimal subsets at build time so broken payloads never reach production.

Evolve APIs through parallel endpoints to maintain contract compliance.

SUMBER DAYA REPOSITORI

Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/26-contract-testing

Commit SHA: e963675c67e7d518e19a07ad930dbdb5be73fff1

Jalankan Demo: go run ./cmd/demo

Jalankan Test: go test -v -race ./...